TASK 2: Use Tokens, Keywords, Identifiers and Constants Correctly

Name: Nagineni Sunil Kumar

Register Number: 192472330

1. Aim

To identify C++ tokens and define metro fares, surcharge values, and balance limits using suitable constants and enumerations.

2. Five Kinds of Tokens

Token Type	Example	Meaning
Keyword	class, int, return	Reserved word in C++
Identifier	SmartCard, balance	Name given to a variable or class
Constant / Literal	1000, 20	Fixed value in the program
Operator	+, -, ==	Performs an operation
Punctuator	{}, ;, ()	Defines program structure

3. Constants in C++

- #define: A preprocessor macro that performs text substitution.
- const: Defines a value that cannot be modified after initialisation.
- constexpr: Defines a value that can be evaluated at compile time when its initialiser permits it.

constexpr is suitable for fixed fare values because it provides type checking and compile-time evaluation.

4. C++ Program

```
#include <iostream>
using namespace std;
enum class Zone {
    Inner,
    Middle,
    Outer };
constexpr long long INNER_FARE = 1000;
constexpr long long MIDDLE_FARE = 2000;
constexpr long long OUTER_FARE = 3000;
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
int main() {
    cout << "Inner fare: " << INNER_FARE << " paise\n";
    cout << "Middle fare: " << MIDDLE_FARE << " paise\n";
    cout << "Outer fare: " << OUTER_FARE << " paise\n";
    cout << "Peak surcharge: "
        << PEAK_SURCHARGE_PERCENT << "%\n";
    cout << "Minimum balance: "
        << MIN_BALANCE_PAISE << " paise\n";
    return 0;}
```

5. Expected Output

Inner fare: 1000 paise

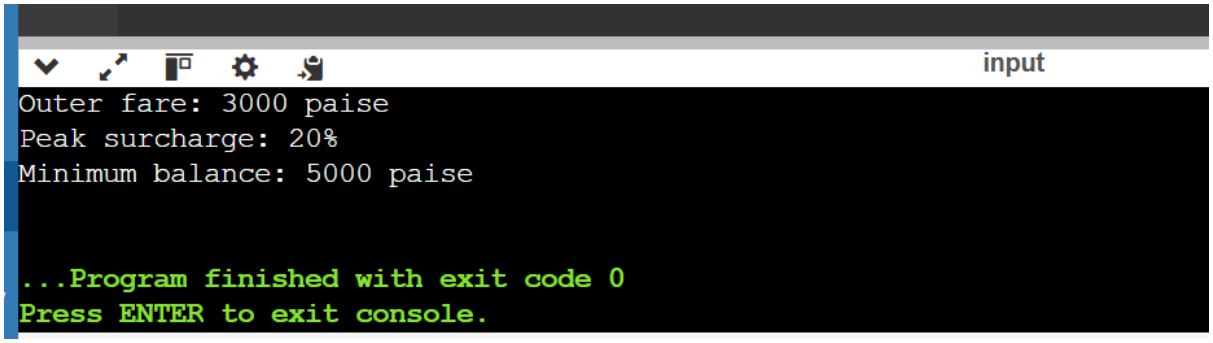
Middle fare: 2000 paise

Outer fare: 3000 paise

Peak surcharge: 20%

Minimum balance: 5000 paise

Obtained output:



6. Why Use enum class?

enum class Zone represents the available zones using named values. It is safer than a traditional enum because its enumerators are scoped and do not implicitly convert to integers.

7. Alternative Concept and Complexity

Using #define macros instead of constexpr values reduces type checking and does not provide normal C++ scope rules. Both approaches typically require no additional runtime storage for simple fixed values, and neither meaningfully changes the time complexity of a fare calculation.

8. Conclusion: Named constexpr constants and enum class improve readability, type safety, and maintainability. A fare revision can be made in one place instead of changing repeated numeric literals throughout the program